Mizar Evo: Design Principles

Reconnecting automated proof, readable mathematics, and verified computation

Kazuhisa Nakasho

Iwate Prefectural University

TPP 2026

RIKEN AIP Tokyo Office

November 16–17, 2026

0.2 - Mizar: early systems, Mizar-rs, and Evo

Table 1. Early Mizar systems and the start of MML

Period	Main milestones
1973–75	Mathematical documents → propositional Mizar-PC (Algol / ODRA)
1977	Mizar-QC: quantifiers (Pascal / CDC 6000)
1982–84	MSE: a smaller language. HPF: mathematical notation and implicit arguments
1986	Mizar-4: multiple passes; ported to PC / DOS at the end of the year
1988–89	PC-Mizar, references between articles; MML began on January 1, 1989

Table 2. PC-Mizar, Mizar-rs, and Mizar Evo

System	Overview
PC-Mizar (Pascal)	Parser → MSM → Analyzer → Checker
Mizar-rs (Rust, ITP 2023)	Reimplements the checker / analyzer to support the existing language **All MML checked:** replaced the checker; the full MML of that time passed **Speed:** about 5–6× faster (8 threads, elapsed time) **Bugs:** four soundness bugs reported; no false MML proofs used them
Mizar Evo (Rust)	New language / processor; keeps mathematical writing and the logical foundation

Source: Matuszewski & Rudnicki (2005); Carneiro (ITP 2023), Table 1.

0.3 - 2023–2026: from an idea to language design

Table 3. Mizar Evo development

Period	Development
Aug 2023	First README: mizar-wishlist
Sep–Oct 2024	Imports, namespaces, structure inheritance, type system
May–Jun 2025	Modes, inference, redefinition; dependent structures / attributes removed and restored
Jan–Apr 2026	EBNF, logical encoding, registration, template and algorithm semantics / contracts
May 2026	Implementation design → test harness → minimal lexer

- **Preparing the 2023 grant proposal changed the plan: from notation extensions to a new language and processor.**
- ITP 2023: Mario Carneiro welcomed a request for implementation help. CICM 2023: wishlist discussion with Adam Naumowicz.
- Early priorities: generic types, namespaces / packages, ATP / traceability. The prototype was to use Mizar-rs.

JSPS KAKENHI, Scientific Research (C), awarded in 2024: 24K14897, "Renewing the language specification of a set-theory-based proof assistant" (title translated).

0.4 - Design development and implementation with AI agents

Developing the language design

- **template:** from generics to a common theorem form for schemes, with structure views and type-argument inference.
- **algorithm:** early requests from Japanese Mizar cryptographers for algorithm proofs, and difficult symbolic calculations in elliptic-curve associativity.

Implementation and discussions since May 2026

- **May: began Rust implementation with AI coding agents. June–July: crate development, agent workflows, SAT-backed checking and audits.**
- **Design → agent implementation → tests and audits → specification revision.**
- September: seminar and discussions with the Mizar team in Białystok.
- September–October: more proof obligations from source; changes to algorithm partiality and contracts.

0.5 - Six challenges and design principles

Table 4. Six challenges and design principles

Area	Challenge in current Mizar	Design principle
§1 Foundation	Keep the logic and MML while replacing the processor	Keep FOL / set theory and a small kernel
§2 Writing	Trace implicit type, registration, and overload choices	Keep mathematical abstractions; explain implicit choices
§3 Generics	Separate mechanisms for definitions, theorems, and schemes	Unify generic mechanisms with templates
§4 Computation	Connect proofs to executable procedures	Check algorithm contracts, invariants, and termination
§5 Tools	Dependencies, distribution, incremental checking, IDE	Modules, namespaces, packages, incremental builds, LSP
§6 Checking	Connect external search to checking evidence	Use ATPs; instantiate evidence and check with SAT

1.1 - Keep the foundation, rebuild the tools

Keep the mathematical language.
Rebuild the processor and development tools.

- **Keep first-order logic (FOL) and Tarski–Grothendieck set theory.**
- Keep soft types, modes, attributes, registrations, structures, and declarative proofs.
- **Separate ATP search from checking: a small kernel independently checks the results. Details in §6.**

1.2 - From sets to topological spaces

Example 1: Cantor's theorem in current Mizar sketch (proof omitted)

```
theorem Cantor:
  for X being set holds card X in card (bool X)
proof ... end;
```

Example 2: a continuous bijection from a compact space to a Hausdorff space (sketch, proof omitted):

```
theorem CompactBijection:
  for S being compact TopSpace,
      T being non empty T_2 TopSpace,
      f being Function of S,T
  st f is continuous one-to-one & rng f = [#] T
  holds f is being_homeomorphism
proof ... end;
```

- **Write mathematics from sets, power sets, and cardinals to topological spaces over first-order set theory.**

2.1 - Trace type, registration, and overload choices

Readable mathematics relies on many automatic decisions.

- **type: which types can this object be used as?** Follow attributes and inheritance. Example: Group $\rightarrow$ Magma, with a unique path.
- **registration: which rules supplied the required attributes?** Example: the chain empty $\rightarrow$ finite $\rightarrow$ countable.
- **overload: which definition and view were selected?** Distinguish definitions with the same notation, or addition and multiplication in a ring.

Keep natural writing and automatic rules, while tracing the reasons for implicit choices.

2.2 - type: declare fields and properties

Structure declarations specification example (inside a definition block)

```
struct LoopStr where
  field carrier -> set;
  field binop -> BinOp of carrier;
  property unit -> Element of carrier;
end;

struct AddLoopStr where
  field carrier -> set;
  field add -> BinOp of carrier;
  property zero -> Element of carrier;
end;
```

A field stores data. A property declares a name and type; its implementation is given separately.

2.3 - type: declare inheritance after the structure

Add a parent mapping after declaring AddLoopStr specification example

```
definition
  inherit AddLoopStr extends LoopStr where
    field carrier from carrier;
    field add from binop;
    property zero from unit;
  end;
end;
```

- **Separate structure declarations from inheritance. This is similar to implementing a Rust trait after defining a type.**
- `from` renames a parent's role: `binop` becomes `add`, and `unit` becomes `zero`.
- One declaration per parent. Equal member types need no proof; narrower types require coherence.

2.4 - type: implement unit for Unital, reuse it as zero

has_unit states that a two-sided identity exists; it does not depend on unit (sketch):

```
definition
  mode UnitalDef: Unital is has_unit LoopStr;
  let M be Unital;
  property M.unit means
    for x being Element of M.carrier holds
      M.binop(it,x) = x & M.binop(x,it) = x;
  existence proof ... end;
  uniqueness proof ... end;
end;
```

- **If A's selected LoopStr view is Unital, reuse its unit implementation as zero.**
- `property zero from unit` gives `A.zero = (A qua LoopStr).unit`. No new zero implementation is needed.

2.5 - registration: automatic chains hide dependencies

Automatic rule chains hide dependencies at the point of use.

```
registration
  cluster EmptyImpliesFinite: empty -> finite for set;
  coherence proof ... end;
  cluster FiniteImpliesCountable: finite -> countable for set;
  coherence proof ... end;
end;
```

Table 5. Registration paths recorded by the new specification

Starting fact	Derived fact	Recorded rule
S is empty	S is finite	EmptyImpliesFinite
S is finite	S is countable	FiniteImpliesCountable

- Declaring `empty` gives `finite` and then `countable` implicitly. No rule citation with `by` is needed.
- **New specification: keep automatic application, and record rule labels and application paths.**

2.6 - overload: two identities in the same ring

In the same ring R, the additive identity is 0 and the multiplicative identity is 1.

- Additive AddMagma: `0 + x = x + 0 = x`
- Multiplicative MulMagma: `1 * x = x * 1 = x`

id. M returns the identity of a unital Magma M sketch (definition omitted)

```
let R be Ring;
id. R;                      :: ambiguous Magma view
id. (R qua AddMagma);        :: R.zero
id. (R qua MulMagma);        :: R.one
```

- Even with one definition of id., the Magma view of R is ambiguous.
- **Use qua to select the additive or multiplicative view and distinguish 0 from 1.**

3.1 - Current MML: more types, more redefinitions and registrations

The sum of functions has one body, but preserving result types needs work for each type.

Table 6. Per-type declarations for function addition in VALUED_1

Value type	Result-type redefinition	Attribute / totality registration
Complex	`PartFunc of C,COMPLEX`	`complex-valued + total`
Real	`PartFunc of C,REAL`	`real-valued + total`
Rational	`PartFunc of C,RAT`	`RAT-valued + total`
Integer	`PartFunc of C,INT`	`INT-valued + total`
Natural	`PartFunc of C,NAT`	`natural-valued + total`

- **Each new type adds redefinitions, registrations, and coherence proofs.**
- We want one template for function addition and its result type.

3.2 - template: share the body and result type

Generic functor sketch (existence and uniqueness proofs omitted)

```
definition
  let T be type extends non empty AddMagma;
  let I be non empty set;
  let f, g be Function of I, T.carrier;
  func AddDef: Add[T,I](f,g) -> Function of I,T.carrier means
    for i being Element of I holds it.i = T.add(f.i,g.i);
  synonym f +[T,I] g for Add[T,I](f,g);
end;
```

Table 7. Template instances and result types

Call (real R / complex C)[1]	Result type
`f + g`	`Function of I,R.carrier`
`u + v`	`Function of I,C.carrier`

- **One definition gives the sum and its result type for each number system.**

[1] Explicit forms: `f +[R,I] g`, `u +[C,I] v`. Omit arguments only when declared types determine them uniquely. Use qua for ambiguous inheritance paths.

Induction as a theorem with a predicate parameter specification example

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

- A predicate parameter describes a family of first-order theorems, one per substituted predicate.[1]
- Explicit instantiation: `defpred`, followed by `by NatInduction[P], Base, Step`.
- Functor parameters are schema-level symbols, not sets. The logic stays first-order.

[1] Details: Białystok deck, Story 6 (`PermProduct[T]` and views with `qua`); also Backup 1 here.

3.4 - template: parameterize structures and attributes too

struct: vector-space data over a field K sketch (inside a definition block)

```
let K be Field;
struct VectorSpace[K] where
  field carrier -> non empty set;
  field add -> BinOp of carrier;
  field smul -> Function of [:K.carrier, carrier:], carrier;
end;
```

`of` / `over` are syntactic sugar (shorter notation): `Vector of K` → `Vector[K]` (a mode example).

attr: a subset S is dense in a space M sketch (condition omitted)

```
let M be MetricSpace;
let S be Subset of M.carrier;
attr DenseDef: S is dense[M] means ...;
```

Prefix attributes are also syntactic sugar: `M-dense` → `dense[M]`.

Type / value parameters are shared across struct, attr, mode, pred, and func.

Apply Hoare rules recursively to generate first-order proof obligations:

{ requires } algorithm body { ensures }

P / Q: precondition / postcondition; B: condition; I: invariant; S: statement.

Table 8. Basic Hoare rules

Syntax	**Conditions to check (partial correctness)**[1]
Assignment `x := e`	`P` $\Rightarrow$ `Q[x` $\leftarrow$ `e]` (replace free x in Q with e)
`if B do S1 else S2 end`	`{P` $\wedge$ `B} S1 {Q}` and `{P` $\wedge$ $\neg$`B} S2 {Q}`
`while B do S end`	Establish: `P` $\Rightarrow$ `I`; preserve: `{I` $\wedge$ `B} S {I}`; exit: `I` $\wedge$ $\neg$`B` $\Rightarrow$ `Q`
`return e`	`P` $\Rightarrow$ `Q[result` $\leftarrow$ `e]`

[1] Basic forms for simple variables and pure expressions. Early return / break / continue and type / domain conditions are omitted.

Euclid's algorithm with a contract specification example

```
terminating algorithm EuclidGcdDef: euclid_gcd(a, b) -> Nat
  requires a >= 1 & b >= 1
  ensures result = Gcd(a, b)
do
  var x := a;  var y := b;
  while y <> 0 do
    invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y);  decreasing y;
    const r := x mod y;  x := y;  y := r;
  end;
  return x;
end;
```

- `terminating` **requires termination for every input satisfying requires. Check the invariant and strict decrease of y.**[1]
- **After verification, promote it to a mathematical functor for formulas and proofs.**

[1] Loop annotations resemble Dafny's `invariant` / `decreases`. Evo uses `invariant` / `decreasing`.

4.3 - Euclid: code and proof obligations

The body again sketch (a,b >= 1; outer context omitted)

```
var x := a; var y := b; :: (1)
while y <> 0 do
  invariant x >= 1 & y >= 0 & Gcd(a,b) = Gcd(x,y); :: (2)
  decreasing y; :: (3)
  const r := x mod y; x := y; y := r;
end;
return x; :: (4)
```

Table 9. Proof obligations for Euclid's algorithm

Point	Proof obligation	Mathematical reason
(1)	Establish the invariant	`x=a`, `y=b`; positivity from the precondition
(2)	Preserve the invariant	`Gcd(x,y)=Gcd(y,r)`; old `y >= 1`, `r >= 0`
(3)	Decrease the Nat measure	`y' = r`, `0 <= r < y`
(4)	Postcondition at exit	`y=0`; `Gcd(a,b) = Gcd(x,0) = x`

Check each point using library lemmas for GCD and remainder.

4.4 - terminating and functor promotion

Table 10. Algorithm termination and guarantees

Algorithm form	Meaning under requires
Without terminating	The contract holds if the call returns
Verified terminating	Total under requires; usable as a functor

Reason with Euclid's contract sketch

```
let a, b be Nat;
assume a >= 1 & b >= 1;
thus euclid_gcd(a,b) = Gcd(a,b) by EuclidGcdDef;
```

Use a computed result in a proof sketch (MVM not implemented)

```
thus euclid_gcd(12,8) = 4 by computation;
```

Run a verified algorithm on MVM, the virtual machine, and use its result in a proof.

The default `computed` path also trusts MVM. It differs from kernel / SAT rechecking.

4.5 - Verified expression rewriting: lambda and differentiation

Write a function directly sketch (u is Real)

```
u |-> (sin u)*u
```

`differentiate` recursively processes expression forms such as products and sin.

Use it in a derivative proof sketch (x is Real; MVM not implemented)

```
thus diff((u |-> (sin u)*u), x) = (cos x)*x + sin x
  by computation(differentiate(u |-> (sin u)*u));
```

- **`|->` denotes a set-theoretic function. An `expr` syntax tree keeps expression structure and variable binding.**
- Verify the contract: on return, the input is differentiable and the result is its derivative.[1]

[1] Assumes differentiation, semantic conversion, and simplification lemmas. This is a specification sketch, not an execution result.

5.1 - Environment: what must an author import?

Current Mizar, shortened from ALGSTR 0 sketch

```
environ
 vocabularies ... STRUCT_0 ...;
 notations ... STRUCT_0;
 constructors ... STRUCT_0 ...;
 registrations ... STRUCT_0;
 theorems STRUCT_0;
```

Table 11. The ten environment directives

Directives	What they import
vocabularies / notations	Symbols / notation
constructors / registrations	Constructors / automatic type facts
requirements / definitions	Special automatic processing / definition information
equalities / expansions	Equality / definition-expansion information
theorems / schemes	Theorems / schemes

- `notations` / `definitions`: article order determines notation / definition priority.
- **Hardest for authors: managing environment overrides and order dependence.**

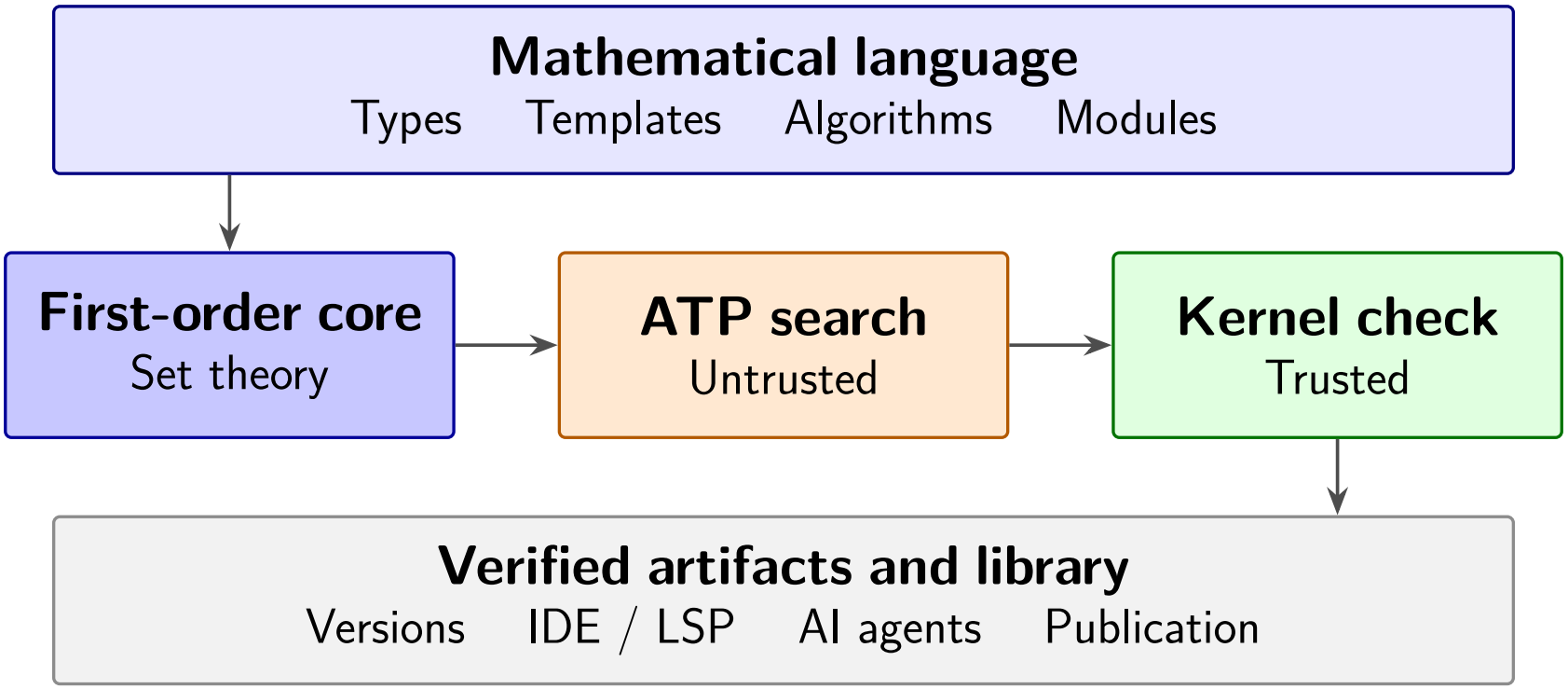
Module imports in the new specification specification example

```
import .function;
import mml.algebra.structure.sorted;
```

Table 12. Imports and dependency tracking

Stage	What authors can inspect
Import	Public definitions, theorems, and registrations
Resolution	Original fully qualified names and registration traces
Reuse	Checked dependency fingerprints for incremental builds

- **Import public items together, and trace what was actually used.**
- The design fixes dependency versions with packages and lockfiles. The IDE shows resolved names and traces.



- **Support rich mathematical writing with a small first-order foundation and modern development tools.**

6.1 - Automation: find lemmas and suggest a proof

Isabelle/HOL: Sledgehammer suggests a proof sketch

```
have "Q a"
  sledgehammer
  by (metis allPQ pa)
```

Mizar Evo: request lemma search with `by atp` (proposed syntax).

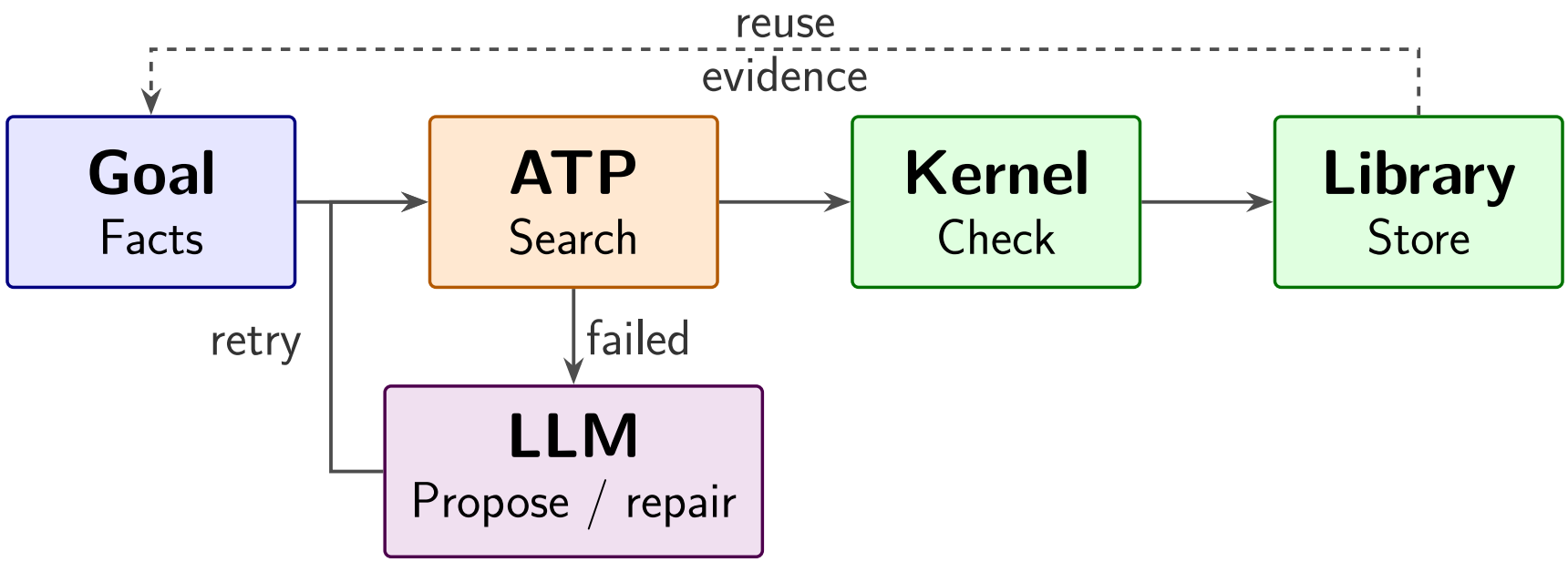
Open goal → library lemma candidates → ATP trials → a `by` suggestion.

Request and suggestion (sketch, planned; Pa is the local assumption P(a)):

```
thus Q(a) by atp;
:: Suggested proof:
:: library.AllPQ: for x holds P(x) implies Q(x)
thus Q(a) by library.AllPQ, Pa;
```

- **Reduce manual lemma search. Keep the selected premises explicit in the source.**
- Apply the suggestion and verify again. The kernel / SAT checks formula and substitution evidence before acceptance.

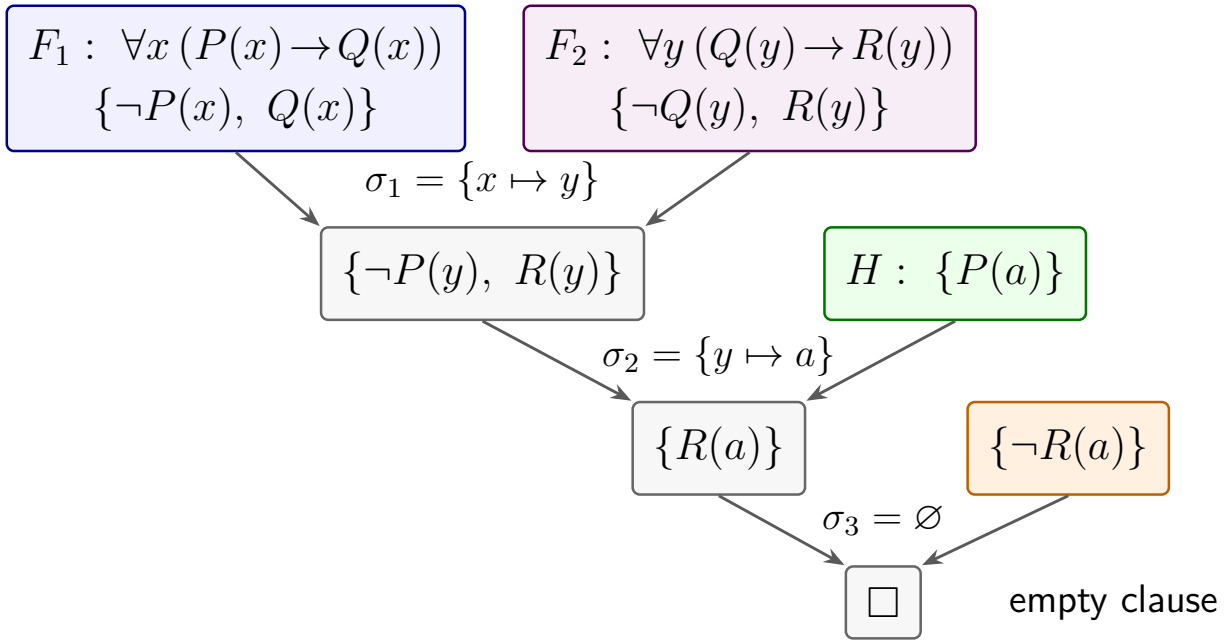
6.2 - Using ATPs: search, check, and reuse



- **Goal and premises → ATP search → kernel checking → library storage and reuse.**
- If unresolved, revise lemmas or strategy and retry. The figure shows design intent.

Getting candidate evidence from a resolution log sketch

Untrusted resolution log (sketch)



Extracted evidence

Compose substitutions along each source branch:

$F_1 : \ x \mapsto y \mapsto a$

$F_2 : \ y \mapsto a$

$\Longrightarrow \ F_1[x := a]$

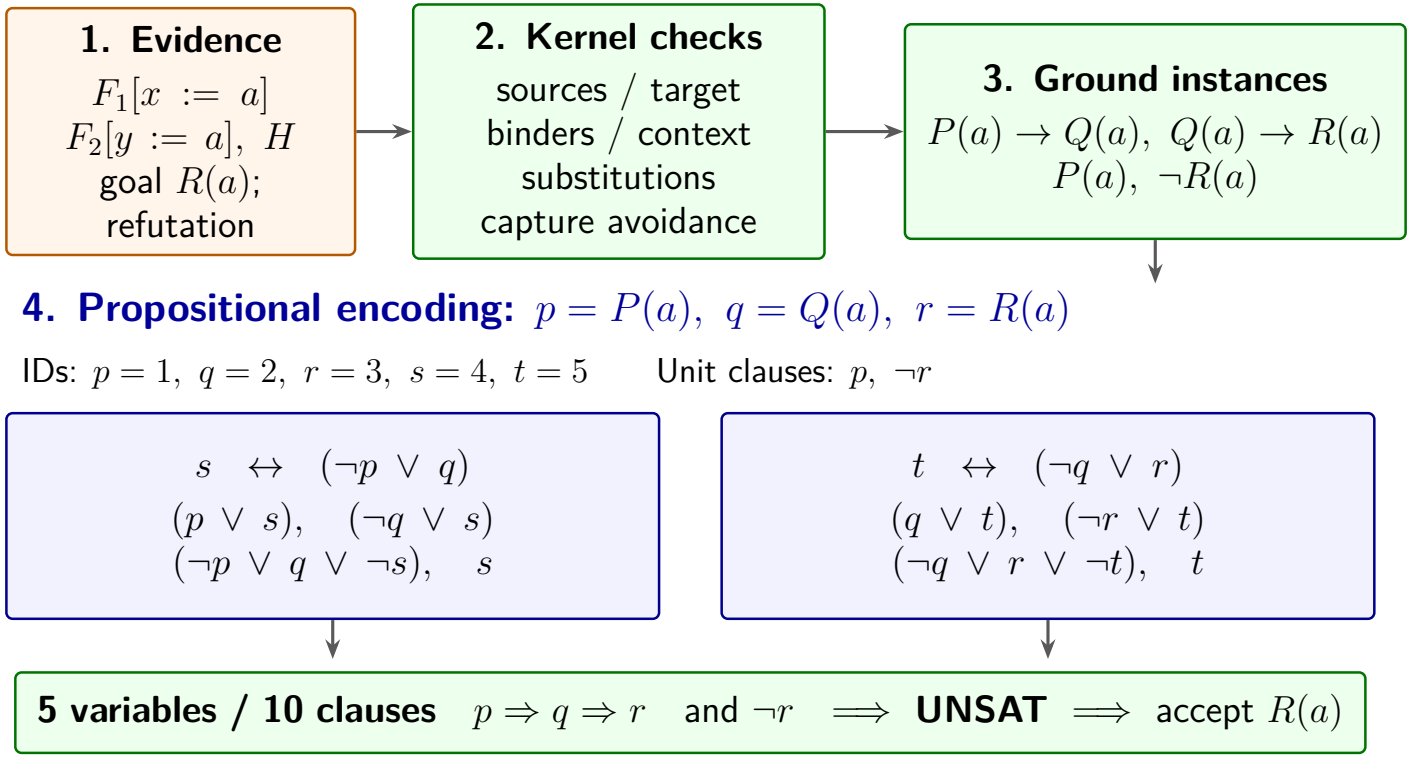
$\Longrightarrow \ F_2[y := a]$

Retain F_1, F_2, H, bindings, context and goal $R(a)$.

The kernel checks evidence; it does not replay this tree.

- **Untrusted extraction: compose to `x := a` for F1 and `y := a` for F2.**

From evidence checks to SAT-based acceptance sketch



- **Check evidence, regenerate CNF, confirm UNSAT with SAT, and accept R(a).**

7.1 - Implementation status and size

Rust product code: 271.1 kLoC; estimated total: 360–540 kLoC (rough estimate).

Table 13. Implementation status and code size (kLoC)

Area	Now	Est. total	Scope / remaining work
lexer / parser / frontend	34.4	36–43	Syntax, source loading, lexing, parsing connected
resolver / checker	122.9	148–193	Limited source semantics; generalization / services remain
Core / VC	49.7	65–95	IR / proof; limited source to VCs
ATP	9.6	16–28	Encoding / candidates; real backend integration remains
kernel	12.8	15–20	Formula / substitution / provenance checks; includes 1.9 for SAT
build / driver / artifact	41.3	56–85	Cache / shared support; full-pipeline publication not connected
LSP	0.5	6–19	Lifecycle / sync; semantic connections remain
MVM / extraction / doc	0	17–57	Runtime / extraction / docs unimplemented

Excluding ATP: kernel + SAT dependencies = 17.6 kLoC.

Core / session counted separately. Excludes dependencies, tests, blank lines, comments. Estimates include current code; rounded.

Source: fixed snapshot 08861708. Counting and estimate basis: `draft.ja.md` §7.

7.2 - Next steps and summary

Next steps

- **First priority: connect source-to-semantics → VC → evidence → kernel → artifact → build.**
- Make algorithm semantics and proof obligations agree; extend the working parts.
- MVM runtime / extraction: separate work. MML porting is separate from processor completion.
- Learned premise selection, LLM repair, cryptography, and quantum algorithms are long-term directions.

Summary

- **Keep FOL, set theory, and mathematical writing; redesign language and tools.**
- **Separate ATP search from trusted checking. The specification is still evolving.**
- **Please share your ITP needs: notation, proof assistance, development tools.**

Backup 1. Template instantiation: one proof, many views

A generic theorem with a bounded type parameter (specification example, proof omitted):

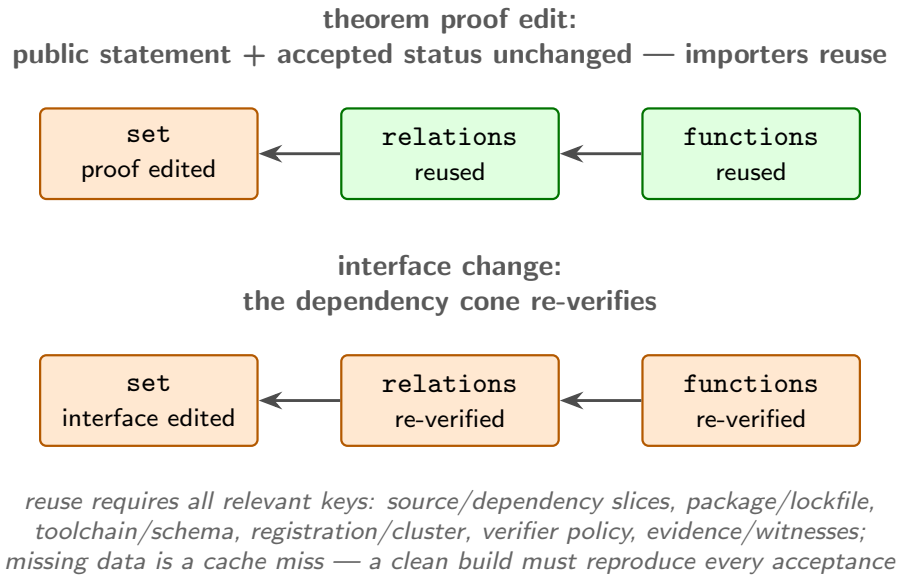
```
definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
        p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

Instances specification example (assuming the required registrations)

```
PermProduct[commutative associative unital AddMagma]
PermProduct[commutative associative unital MulMagma]
let R be commutative Ring;  PermProduct[R qua AddMagma]   :: additive view
```

- A ring has two inheritance paths to Magma. The view selected by qua determines the notation.

Backup 2. Incremental verification



- A proof-body edit needs no rebuild of other modules if the public claim and acceptance status stay unchanged. An interface change rechecks all direct and indirect dependents.
- Reuse needs exact matches for all relevant cache keys. Missing data means a miss. A clean build must reproduce every accepted result; reuse alone does not prove correctness.

Backup 3. Package manifest

Mizar Evo specification example

```
[package]
name    = "algebra"
version = "2.3.1"
edition = "2025"

[dependencies]
mml_core = "^1.0"
topology = { version = "^0.9", features = ["metric"] }
```

- Reproducible builds need fixed sources, lockfile, toolchain, checker configuration, and deterministic ATP evidence.
- Versions let authors reuse articles without manual copies between libraries. One module can collect a field's public items into a single import.

Source: `doc/spec/en/23.package_management_and_build_system.md`; Białystok deck, Story 1.

Backup 4. Comparing evaluation conditions

Table 14. MizAR 60 and Sledgehammer evaluation conditions

	MizAR 60 (ITP 2023)	**Sledgehammer / AFP (ITP 2022)**
Reported success	58.4% (1,690 / 2,896)	68.8% (3,440 / 5,000)
Evaluation set	MML 1147 theorems / lemmas; 2,896 holdout problems out of 57,897	5,000 local goals from 50 AFP entries
Premise selection	Learned selection from the whole library	MePo; baseline 512 facts, varied by configuration
Time budget	Portfolio total: 420 CPU seconds	Greedy 16 configurations $\times$ 30 CPU seconds = 480 CPU seconds
Success criterion	ATP proof found (hammering mode)	External-prover proof found; Isabelle reconstruction excluded

- **Different targets, premise selection, time budgets, and configuration choices. Success rates alone cannot rank the designs.**

Outline of encoding steps sketch

```
F X                    ->  app(F, X)
(%x. t) ...            ->  fresh constant + defining axioms   (lambda lifting)
polymorphic types      ->  type guards or type tags
Boolean-valued terms   ->  extra encoding
```

- **Use `app` for function-variable application. Encode lambdas with lambda lifting or combinators, and types with guards or tags.**
- **Reconstruct the ATP proof inside Isabelle: `metis`, `smt` replay, or generated Isar text.**
- These techniques connect higher-order ITPs to first-order ATPs.
- This explains the connection, not a measured performance advantage.